

upfbench — Framework Overview & Architecture

Portable test framework for 5G UPFs

coRAN Labs

June 2026

Contents

1. What it is (in one paragraph)	1
2. Why it exists (the gap)	1
3. The big picture (data flow)	2
4. The core design idea — swappable plugins	2
5. The three suites (what the user picks)	3
6. Directory structure — what every part means	3
7. How one test flows end-to-end (concrete walkthrough: TC-01 throughput) . .	5
8. Key design decisions	5
9. Current status (validated end-to-end)	5
10. How to extend it	6
11. Glossary (for readers newer to 5G UPF internals)	6

A guide for the team: what this framework is, why it exists, the three test suites, how it's built, and what every part of the codebase does.

For setup/run instructions see [RUNBOOK.md](#); for the formal plan and roadmap see [PLAN.md](#) and [HANDOFF.md](#).

1. What it is (in one paragraph)

upfbench is a portable test framework for 5G User Plane Functions (UPFs). You point it at a UPF, choose a **test suite**, and it drives the UPF (traffic + PFCP control), measures it from the UPF's own counters, and produces a **standards-aligned report** (RFC 2544 / RFC 8219 / ETSI NFV-TST 009 / 3GPP TS 29.244). The first target is the **SD-Core BESS-UPF**; the design lets you add other UPFs (OAI, Open5GS, free5GC, eUPF) by writing one small plugin — without touching the tests.

2. Why it exists (the gap)

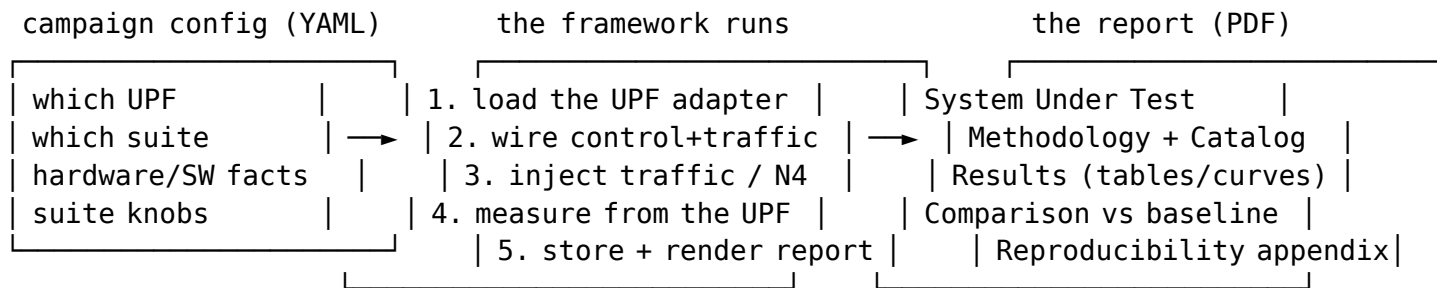
A UPF has three things worth testing: **data-plane performance**, **multi-UE scale**, and **N4/PFCP correctness**. Existing tools each cover only a slice:

- omec/upf PTF — close, but **BESS-locked** (bypasses PFCP).

- pfcpsim — **N4/PFCP only** (no data-plane performance).
- VSPERF / NFVbench — portable performance, but **vSwitch-generic** (no GTP-U/PFCP awareness).
- Per-UPF harnesses (eUPF, upf-bpf) — **single implementation**.

upfbench is the missing piece: **one UPF-agnostic framework** that unifies all three axes over the standardized **N3 (GTP-U)** and **N4 (PFCP)** interfaces. It also automates the manual benchmark methodology coRAN Labs already ran by hand, and generalizes the “four ways to move a packet” study (AF_PACKET / AF_XDP / CNDP / DPDK).

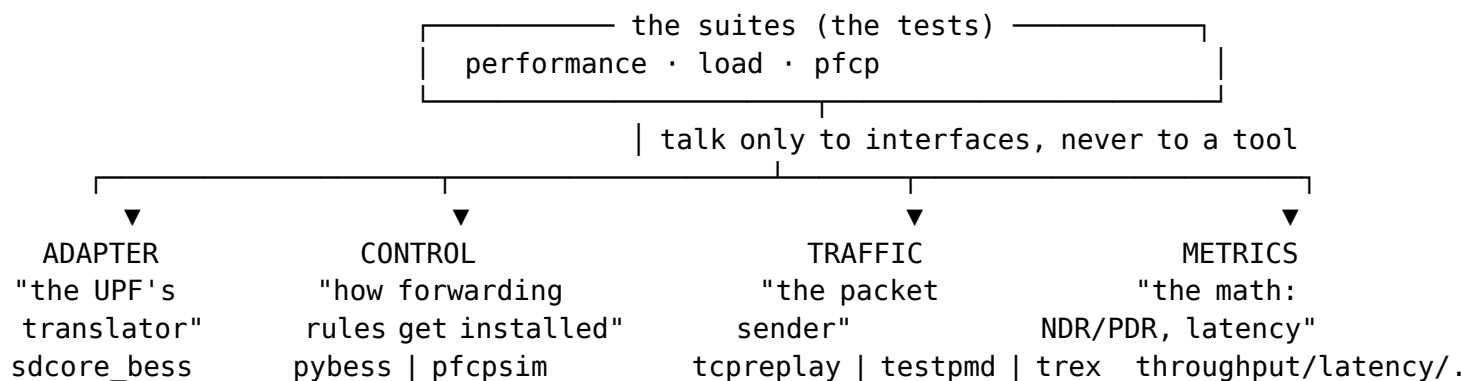
3. The big picture (data flow)



One input (a YAML campaign file), five steps inside the **runner**, and a per-suite report out. Everything in the middle is built from **swappable plugins** (next section).

4. The core design idea — swappable plugins

The framework never talks to a specific UPF, generator, or control protocol directly. It talks to **four plugin interfaces**, and the concrete tools plug in behind them. This is what makes it portable: to support a new UPF you write one new **adapter**; the suites, traffic, and report code don’t change.



- **Adapter = the middleman/translator for one UPF.** The rest of the framework only knows “an adapter”; the adapter knows how to talk to this specific UPF (read its facts, read its packet counters). Swap UPF = swap adapter.
- **Control = how you tell the UPF to forward.** Either the BESS-specific fast path (pybess) or the standardized PFCP path (pfcpsim).
- **Traffic = the thing that sends packets.** On this VM that’s tcpreplay (GTP-U); testpmd/trex are the real-NIC/DPDK options.

- **Metrics = the measurement helpers** (e.g., the RFC 2544 NDR/PDR binary search).

5. The three suites (what the user picks)

Suite 1 — performance (single tunnel, maximum rate)

The classic data-plane benchmark. One flow, pushed as hard as possible. - **TC-01** Throughput — NDR (no-drop rate) and PDR ($\leq 0.1\%$ loss) per frame size, via a binary-search of the offered rate (RFC 2544). - **TC-03** Latency / jitter — measured inside the UPF pipeline (avg/p50/p99/p99.9). - **TC-04** Burst — saturating blast; checks for drops and clean drain. - **TC-08** Multi-flow — many flows at once (does flow diversity change the rate?). - **TC-02** Bidirectional — deferred (needs a real downlink path). - **Tools:** pybess (install a “forward-everything” rule) + tcpreplay (send GTP-U).

Suite 2 — load (many UEs at once, UPF-isolated)

Tests scale without needing a full RAN. pfcpsim installs **N real per-UE PFCP sessions** (one PDR/FAR/QER set each); traffic is driven on **N matching per-UE TEIDs**. - **LT-01** Session capacity — how many sessions the UPF accepts (the ceiling) + install rate. - **LT-02** Per-UE throughput — aggregate forwarded under N UEs, plus a per-UE forwarding verification (confirms each UE’s session actually forwards). - **LT-03** Latency under load — how latency degrades as the UE count rises. - **Tools:** pfcpsim (sessions) + tcpreplay (per-TEID GTP-U).

Suite 3 — pfcpc (N4 conformance, 3GPP TS 29.244)

Does the UPF’s control plane behave correctly? pfcpsim drives each N4 procedure and the framework asserts the response — output is a **pass/fail matrix**. - **CF-01** association setup/release · **CF-02** session establishment · **CF-03** session modification · **CF-04** session deletion · **CF-05** error handling (unknown-SEID rejection). - **Tools:** pfcpsim only (pure control plane — no traffic generator).

Dependency insight: Suite 1 uses the BESS fast path; Suites 2 **and** 3 both ride on pfcpsim, so integrating pfcpsim unlocked two suites at once.

6. Directory structure — what every part means

upf-benchmark-framework/

— README.md	← one-page intro + status
— pyproject.toml	← Python package metadata + deps (PyYAML, Jinja2)
— configs/	
— sdcore-bess.yaml	← THE INPUT: a "campaign" — which UPF, which suite, all knobs
— docs/	← PLAN (design), HANDOFF (onboarding), RUNBOOK (how to run), this OV
— third_party/	
— pfcpsim/	← vendored omec-project PFCP simulator (Go source) —
reused, not rewritten	

- └─ campaigns/ ← OUTPUTS: one folder per run (results.json, report-*.pdf, raw/) – gitignored
- └─ upfbench/ ← the framework itself (Python)
 - └─ cli.py ← the command-line entry: `list`, `run --config ... --suite ...`, interactive menu
 - └─ config.py ← loads + validates the campaign YAML into typed objects
 - └─ runner.py ← THE ORCHESTRATOR: the 5-step loop (load adapter → wire plugins → m
 - └─ search.py ← the RFC 2544 NDR/PDR binary-
- search algorithm (generator-agnostic)
 - └─ results.py ← the campaign store: collects results → writes results.json (+ cap
 - └─ adapters/ ← THE PER-UPF PLUGIN ("the middleman/translator")
 - └─ base.py ← the contract every UPF must satisfy (describe(), port_counters(
 - └─ sdcore_bess.py ← the SD-Core BESS-UPF implementation (talks to the pod via kube
 - └─ control/ ← HOW FORWARDING RULES / SESSIONS GET INSTALLED
 - └─ base.py ← the control contract (setup, install_sessions, modify, delete,
 - └─ pybess.py ← BESS white-box fast path (wildcard forward-
 - all rule) – Suite 1
 - └─ pfcpsim.py ← standardized PFCP/N4 via pfcpsim – Suites 2 & 3
 - └─ traffic/ ← THE PACKET SENDERS (traffic generators)
 - └─ base.py ← the generator contract (run_trial() → a Trial with rates)
 - └─ tcpreplay.py ← host-side GTP-U injector (what this VM uses) –
 - crafts pcap, replays at a rate
 - └─ testpmd.py ← DPDK testpmd (real-NIC raw frames) – stub, real-
 - hardware path
 - └─ trex.py ← Cisco TRex (GTP-U, per-flow stats) – stub, DPDK-
 - hardware path
 - └─ metrics/ ← MEASUREMENT HELPERS (the "math" layer)
 - └─ throughput.py ← NDR/PDR helpers (built on search.py)
 - └─ latency.py ← latency/jitter helpers
 - └─ sessions.py ← per-UE / capacity accounting (Suite 2)
 - └─ resource.py ← CPU/mem / per-core efficiency (future)
 - └─ suites/ ← THE TESTS THEMSELVES (one folder per suite)
 - └─ base.py ← the TestCase contract + RunContext (the wired-
 - up plugins handed to each test)
 - └─ registry.py ← discovers the test cases + says which plugins each suite needs
 - └─ performance/ ← Suite 1: tc01_throughput, tc02_bidirectional, tc03_latency, t
 - └─ load/ ← Suite 2: lt01_session_capacity, lt02_throughput_per_ue, lt03_la
 - and _flows.py (computes per-
 - UE TEID/IP that match pfcpsim's sessions)
 - └─ pfcpsim/ ← Suite 3: cf01..cf05 (the conformance checks)
 - └─ report/ ← TURNS RESULTS INTO A DOCUMENT

— render.py	← fills the right template per suite, compiles to PDF (or .tex), a
— templates/	← one LaTeX/Jinja template per suite (performance / load / pfcps)

The mental model: config is the order form → runner is the kitchen → adapter, control, traffic, metrics are the appliances → suites are the recipes → report is the plated dish. Adding a new UPF is like adding one new appliance (adapter); the recipes and plating don't change.

7. How one test flows end-to-end (concrete walkthrough: TC-01 throughput)

1. **CLI** parses `run --config sdcore-bess.yaml --suite performance`.
2. **config.py** loads the YAML → a Campaign object.
3. **runner.py** loads the `sdcore_bess` **adapter**, reads live SUT facts (image, mode, ports) into the report, then wires the pybess **control** + tcpreplay **traffic** the suite asks for.
4. **control** installs the wildcard “forward-everything” rule on the UPF.
5. The **TC-01 test** loops over frame sizes. For each, it calls `search.binary_search`, which repeatedly asks the **traffic** generator to send at a target rate; after each trial the test reads the **adapter's** port counters to compute loss; the search converges on NDR (0 loss) and PDR ($\leq 0.1\%$).
6. The test returns a TestResult (a table + KPIs); **results.py** stores it.
7. **render.py** fills `performance.tex.j2` and compiles `report-performance.pdf`.

Every command the framework runs is captured into the report's **reproducibility appendix**.

8. Key design decisions

- **Python** — pybess, the TRex API, and Scapy are all Python.
- **Reuse, don't reinvent** — pfcpsim (spec-correct PFCP) and tcpreplay/testpmd/TRex (traffic) are wrapped, not rewritten. Our original code is the thin glue layer.
- **Plugins behind interfaces** — adapter / control / traffic / metrics are swappable; the suites only see the interfaces. This is the portability guarantee.
- **The report template is the spec** — the framework's job is to fill it in automatically.
- **Connect-only by default** — the framework attaches to an already-running UPF; it does not deploy it.

9. Current status (validated end-to-end)

All three suites work against SD-Core BESS-UPF in **af_packet** mode and render PDF reports. Honest caveats: numbers are af_packet on a single shared VM (absolute throughput ~1/5 of a dedicated isolated-core server, but the methodology and qualitative behavior match the reference); throughput is noisy at af_packet's low rates (longer trials stabilize it). See RUNBOOK.md §9 and HANDOFF.md §10 for the remaining roadmap (TC-02 bidirectional, true per-UE fairness, worker auto-tune, more UPF adapters, TRex generator).

10. How to extend it

- **Add a new UPF** → write `upfbench/adapters/<name>.py` with a class `Adapter` implementing `describe()` + `port_counters()`. Point the config's `upf.adapter` at it. Nothing else changes.
- **Add a traffic generator** → write `upfbench/traffic/<name>.py` with a class `Generator` implementing `run_trial()`. Select it via the config's per-suite generator.
- **Add a test** → drop a `TestCase` subclass into the suite folder and list it in that module's `TESTS`. The registry picks it up automatically.

11. Glossary (for readers newer to 5G UPF internals)

- **UPF** — User Plane Function: the 5G core's data-plane node that forwards user traffic.
- **N3 / N4 / N6** — UPF interfaces: N3 = toward the gNB (GTP-U tunnels), N6 = toward the internet/data network, N4 = control (PFCP) from the SMF.
- **GTP-U** — the tunneling protocol on N3; each UE's traffic rides a tunnel identified by a **TEID**.
- **PFCP** — the N4 control protocol (3GPP TS 29.244); the SMF uses it to program the UPF.
- **PDR / FAR / QER** — the per-UE forwarding rules the SMF installs: Packet Detection Rule (match), Forwarding Action Rule (what to do), QoS Enforcement Rule (rate/marketing).
- **NDR / PDR (benchmark)** — No-Drop Rate / Partial-Drop Rate: the RFC 2544 throughput metrics (highest offered rate with 0% / $\leq$ tolerance loss).
- **af_packet / AF_XDP / CNDP / DPDK** — the UPF data-plane I/O modes (kernel-socket → kernel-bypass), in increasing performance. This VM runs the simplest, **af_packet**.
- **bessd / pybess / bessctl** — the BESS software dataplane that powers SD-Core's UPF, its Python gRPC API, and its CLI.
- **SUT** — System Under Test (the UPF + its host, recorded in the report).